[VHH Project] Pipeline Prototyping

July, 2023

This document is meant to re-build an Immcantation (https://immcantation.readthedocs.io/) B-cell repertoire analysis, and relies completely on documentation from within that framework.

Objectives:

Refactor and write documentation to generalize for future use

- Part 1: (BASH) Raw Read Assembly (pRESTO)
- Part 2: (BASH) Change-O - Repertoire Clonal Assignment Toolkit
- Part 3: (R) Spectral Clustering-Based Method for Identifying B Cell Clones (Scoper)
- Part 4: (BASH) Reconstruct Germlines and Build Phylogenetic Trees
- Part 5: (R) Create an alpaca SHM Model

Part 1) pRESTO Prototype Workflow

Note that this document relies completely on Illumina MiSeq 2x250 BCR mRNA example workflow (https://presto.readthedocs.io/en/stable/workflows/Greiff2014_Workflow.html) from pRESTO (The REpertoire Sequencing TOolkit).

Data Sources

Focuses on data from: Greiff, V. et al. “Quantitative assessment of the robustness of next-generation sequencing of antibody variable gene repertoires from immunized mice.” *BMC Immunol.* (2014)

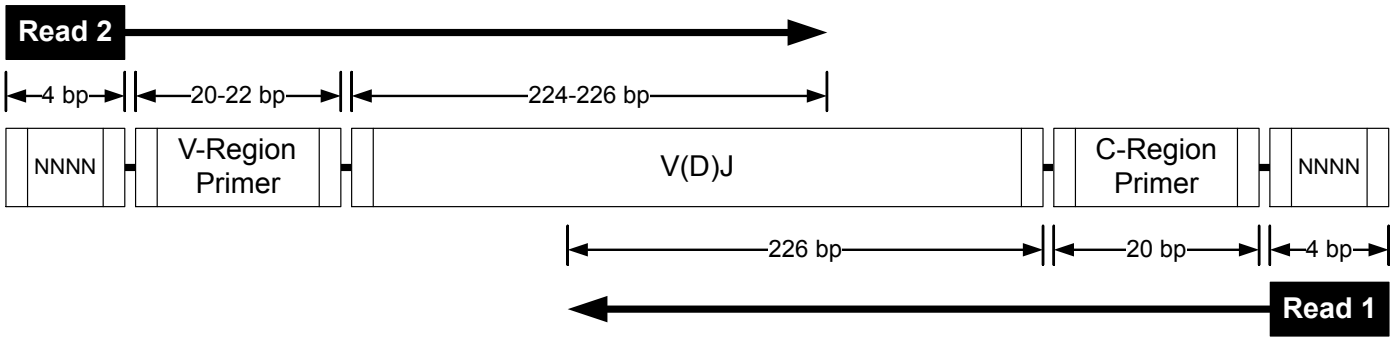
Overview

Here, we’re going to review each step required to translate raw sequence reads to an annotated repertoir of V(D)J sequences.

Overall, this is divided into:

1. Paired-end assembly
2. Quality control and primer annotation
3. Deduplication and filtering

Read Configuration



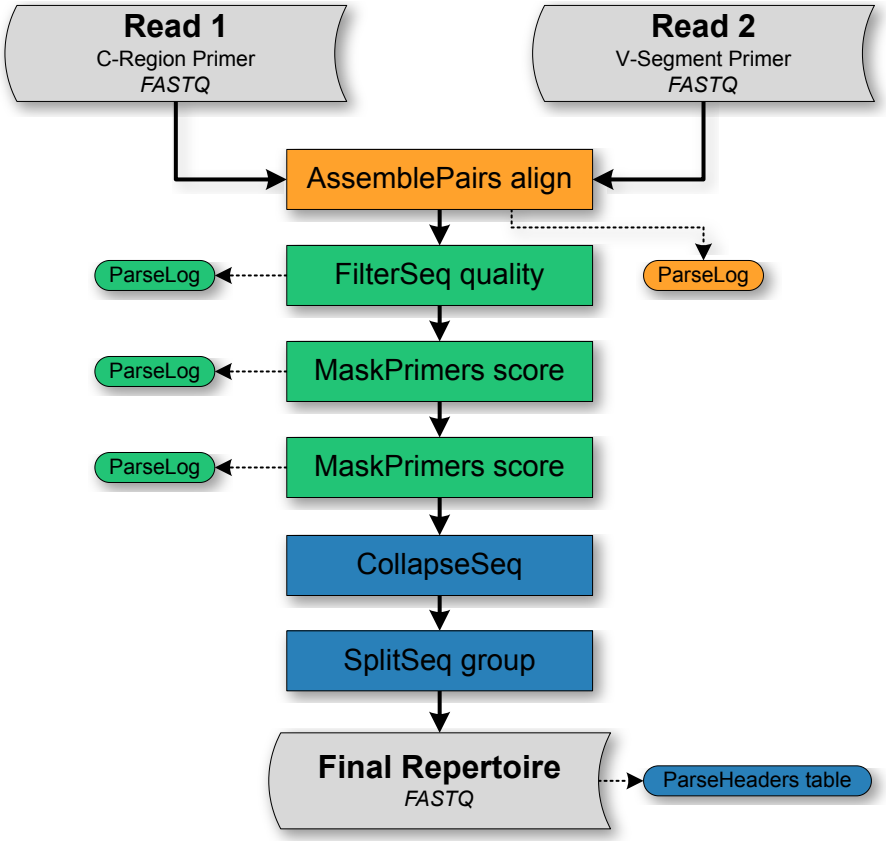
Schematic of Illumina MiSeq 2x250 stranded paired-end reads without UMI barcodes.

Notes (per workflow documentation):

*Each 250 base-pair read was sequenced from one end of the target cDNA, so that the two reads together cover the entire variable region of the Ig heavy chain.

- The V(D)J reading frame proceeds from the start of read 2 to the start of read 1.
- Read 1 is in the opposite orientation (reverse complement), and contains the C-region primer sequence.
- Both reads begin with a random sequences of 4 nucleotides.

FlowChart



Installation and Environment Setup

Immcantation

- First, immcantation v.4.4.0 (https://hub.docker.com/r/immcantation/suite) was downloaded via Docker

```
docker pull immcantation/suite:4.4.0
```

- Next, the SRA Toolkit (https://github.com/ncbi/sra-tools/wiki/01.-Downloading-SRA-Toolkit) was downloaded, and the installation guide (https://github.com/ncbi/sra-tools/wiki/02.-Installing-SRA-Toolkit) followed.

SRA Toolkit

We need the first 25,000 sequences of sample `ERR346600`, which may downloaded using fastq-dump from the SRA Toolkit (https://github.com/ncbi/sra-tools/wiki/01.-Downloading-SRA-Toolkit).

```
fastq-dump --split-files -X 25000 ERR346600
```

1) Paired-end assembly

align

Each set of paired-ends mate-pairs is first assembled into a full length Ig sequence using the align subcommand of `AssemblePairs.py`:

During assembly we have defined read 2 (V-region) as the head of the sequence (-1) and read 1 as the tail of the sequence (-2). The `--coord` argument defines the format of the sequence header so that AssemblePairs.py can properly identify mate-pairs; in this case, we use `--coord sra` as our headers are in the SRA/ENA format.

Our Command

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  AssemblePairs.py align -1 /data/ERR346600_2.fastq -2 /data/ERR346600_1.fastq \
    --coord sra --rc tail --maxerror 0.1 --outname M1 --log /data/AP.log
```

Output:

1. `M1_assemble-pass.fastq`
2. `AP.log`

ParseLog.py

The `ParseLog.py` tool is then used to build a tab-delimited file of results from the `AssemblePairs.py` log file:

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  ParseLog.py -l /data/AP.log -f ID LENGTH OVERLAP ERROR PVALUE
```

Output:

1. `AP_table.tab` (tsv)

Which will containing the following columns:

```
Field Description

ID - Sequence name

LENGTH - Length of the assembled sequence

OVERLAP - Length of the overlap between mate-pairs

ERROR - Mismatch rate of the overlapping region

P-VALUE - P-value for the assembly
```

2) (Quality Control Step) Filter Sequence

Removing Low-Quality Reads

Quality control begins with the identification and removal of low-quality reads using the quality subcommand of the `FilterSeq.py` tool. In this example, reads with mean Phred quality scores less than 20 (`-q 20`) are removed:

Tutorial Command

```
FilterSeq.py quality -s M1_assemble-pass.fastq -q 20 --outname M1 --log FS.log
```

TH Command

Throw out low quality reads, anything below Q30 gets tossed

```
sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    FilterSeq.py quality -s $SEQ1 -q 30 -o $SEQ2
```

Our Command

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  FilterSeq.py quality -s /data/M1_assemble-pass.fastq -q 20 --outname M1 --log /data/F
S.log
```

Output:

1. `M1_quality-pass.fastq`
2. `FS.log`

ParseLog.py

The `ParseLog.py` tool is then used to build tab-delimited file from the `FilterSeq.py` log:

Our Command

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  ParseLog.py -l /data/FS.log -f ID QUALITY
```

Output:

1. `FS_table.tab`

With the following variables:

```
Field Description

ID - Sequence name

QUALITY - Quality score
```

Read annotation and masking of primer regions

When dealing with Ig sequences, it is important to cut or mask the primers, as B cell receptors are subject to somatic hypermutation (the accumulation of point mutations in the DNA) and degenerate primer matches can look like mutations in downstream applications. The `score` subcommand of `MaskPrimers.py` is used to identify and remove the V-segment and C-region PCR primers for both reads:

Tutorial Command

```
MaskPrimers.py score -s M1_quality-pass.fastq -p Greiff2014_VPrimers.fasta \
    --start 4 --mode mask --pf VPRIMER --outname M1-FWD --log MPV.log

MaskPrimers.py score -s M1-FWD_primers-pass.fastq -p Greiff2014_CPrimers.fasta \
    --start 4 --mode cut --revpr --pf CPRIMER --outname M1-REV --log MPC.log
```

In this data set the authors have added a random sequence of 4 bp to the start of each read before the primer sequence to increase sequence diversity and the reliability of cluster calling on the Illumina platform. As such, both primers begin at position 4 (`--start 4`), but the C-region primer begins 4 bases from the end of the assembled read. The addition of the `--revpr` argument to the second `MaskPrimers.py` step instructs the tool to reverse complement the primer sequences and check the tail of the read.

The two primer regions have also been treated differently. The V-segment primer has been masked (replaced by Ns) using the `--mode mask` argument to preserve the V(D)J length, while the C-region primer has been removed from the sequence using the `--mode cut` argument.

During each iteration of the `MaskPrimers.py` tool an annotation field was added with the name of the primer, with the name of the field specified by the `--pf` argument, such that after both iterations each sequence contains an annotation of the form:

TH Command

```
#match the reverse primers and read the reverse half of the barcode
sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    MaskPrimers.py score -s $SEQ3 -p data/$Sub0/AlpVHH.fasta --barcode \
        --start 8 --maxerror 0.01 --mode cut --revpr --pf CPRIMER \
        -o $SEQ4

#match the forward primers and read the forward half of the barcode
sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    MaskPrimers.py score -s $SEQ2 -p data/$Sub0/AlpVHH.fasta --barcode \
        --start 8 --maxerror 0.01 --mode cut --pf VPRIMER \
        -o $SEQ3
```

Our Commands

Note that we will change the max error rate, because no sequences are passing..

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  MaskPrimers.py score -s /data/M1_quality-pass.fastq -p /data/Greiff2014_VPrimers.fasta
\
      --start 4 --mode mask --pf VPRIMER --outname M1-FWD --log /data/MPV.log \
      --maxerror 0.50
```

Output:

1. `M1-FWD_primers-pass.fastq`
2. `MPV.log`

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
MaskPrimers.py score -s /data/M1-FWD_primers-pass.fastq -p /data/Greiff2014_CPrimers.fas
ta \
    --start 4 --mode cut --revpr --pf CPRIMER --outname M1-REV --log /data/MPC.log \
    --maxerror 0.50
```

Output:

1. `M1-REV_primers-pass.fastq`
2. `MPC.log`

Parse Log Summary

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  ParseLog.py -l /data/MPV.log /data/MPC.log -f ID PRIMER ERROR
```

Output:

1. `MPC_table.tab`

Capturing the following annotations:

```
Field Description

ID Sequence name

PRIMER Primer name

ERROR Primer match error rate
```

Example note:

This library was prepared in a stranded manner. Meaning, the read orientation is constant for all reads; read 1 is always the C-region end of the amplicon and read 2 is always the V-segment end. If your data is unstranded (50% of the reads are forward, 50% are reversed), then you must modify the first MaskPrimers.py step to account for this by using the align subcommand instead:

```
MaskPrimers.py align -s M1*quality-pass.fastq -p Greiff2014_VPrimers.fasta \
    --maxlen 30 --mode mask --pf VPRIMER --log MP1.log
```

This will perform a slower process of locally aligning the primers, checking the reverse compliment of each read for matches, and correcting the the output sequences to the forward orientation (V to J).

3) Deduplication and Filtering

Removal of duplicate sequences

The last stage of the workflow involves two filtering steps to yield the final repertoire.

First, the set of unique sequences is identified using the `CollapseSeq.py` tool, allowing for up to 20 interior N-valued positions (`-n 20` and `--inner`), and requiring that all reads considered duplicates share the same C-region primer annotation (`--uf CPRIMER`). Additionally, the V-segment primer annotations of the set of duplicate reads are propagated into the annotation of each retained unique sequence (`--cf VPRIMER` and `--act set`):

Tutorial Command

```
CollapseSeq.py -s M1-REV_primers-pass.fastq -n 20 --inner --uf CPRIMER \
    --cf VPRIMER --act set --outname M1
```

TH Command

```
NOTE: TH moves directly to:

#concatenate the barcodes from each end to form full unique identifiers
sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseHeaders.py collapse -s $SEQ4 -f BARCODE --act cat -o $SEQ5

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    BuildConsensus.py -s $SEQ5  --pf VPRIMER \
         --maxerror 0.1  -o $SEQ6 -n 2 --bf BARCODE
```

Our Command

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  CollapseSeq.py -s /data/M1-REV_primers-pass.fastq -n 20 --inner --uf CPRIMER \
    --cf VPRIMER --act set --outname M1
```

Filtering to repeated sequences

CollapseSeq stores the count of duplicate reads for each sequence in the `DUPCOUNT` annotation. Following duplicate removal, the data is subset to only those unique sequence with at least two representative reads by using the `group` subcommand of `SplitSeq.py` on the count field (`-f DUPCOUNT`) and specifying a numeric threshold (`--num 2`):

Tutorial Command

```
SplitSeq.py group -s M1_collapse-unique.fastq -f DUPCOUNT --num 2 --outname M1
```

Our Command

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  SplitSeq.py group -s /data/M1_collapse-unique.fastq -f DUPCOUNT --num 2 --outname M1
```

Creating the annotation table

We now generate annotations:

1. `DUPCOUNT` - Duplicate read count
2. `CPRIMER` - Isotype
3. `VPRIMER` - B-segment primer

These are generated from the final repertoire data from the `SplitSeq.py` output with the `table` subcommand from `ParseHeaders.py` :

```
docker run -v /Users/emoseley/Desktop/Tufts/Su23-DS-288-Capstone/edwardmoseley.github.i
o/code/data:/data immcantation/suite:4.4.0 \
  ParseHeaders.py table -s /data/M1_collapse-unique.fastq -f ID DUPCOUNT CPRIMER VPRIMER
```

Output:

1. `M1_collapse-unique_headers.tab`

Final Results

The final set of sequences, which serve as input to a V(D)J reference aligner (Eg, IMGT/HighV-QUEST or IgBLAST), and tables that can be plotted for quality control are:

1. `M1_collapse-unique.fastq` - Total Unique Sequences
2. `M1_atleast-2.fastq` - Unique sequences represented by at least 2 reads

3. `M1_atleast-2_headers.tab` - Annotation table of the `atleast-2` file
4. `AP_table.tab` - Table of the AssemblePairs log
5. `FS_table.tab` - Table of the FilterSeq log
6. `MPV_table.tab` - Table of the V-segment MaskPrimers log
7. `MPC_table.tab` - Table of the C-region MaskPrimers log

Part 2) Change-O - Repertoire Clonal Assignment Toolkit

Here, we rely on the IgBLAST (https://changeo.readthedocs.io/en/stable/examples/igblast.html) tutorial, within the Change-O - Repertoire clonal assignment toolkit (https://changeo.readthedocs.io/en/stable/examples/igblast.html), and we use an example data set from the UMI barcoded MiSeq Workflow (https://presto.readthedocs.io/en/stable/workflows/Stern2014_Workflow.html). The data can be downloaded from:

- Change-O Example Files (http://clip.med.yale.edu/immcantation/examples/AIRR_Example.tar.gz)

TH Commands

```
%%bash

WD=/home/troyhinkley/ncbi-igblast-1.17.1
DATA=/home/troyhinkley/alpaca/igblast

# V segment database
$WD/bin/edit_imgt_file.pl \
$DATA/vdj_gapped/imgt_gapped_alpaca_IGHV.fasta > $WD/internal_data/alpaca/imgt_alpaca_ig
_v.fasta


$WD/bin/makeblastdb -parse_seqids -dbtype nucl \
    -in $WD/internal_data/alpaca/imgt_alpaca_ig_v.fasta \
    -out $WD/internal_data/alpaca/alpaca_V


# D segment database
$WD/bin/edit_imgt_file.pl \
$DATA/vdj_gapped/imgt_gapped_alpaca_IGHD.fasta > $WD/internal_data/alpaca/imgt_alpaca_ig
_d.fasta

$WD/bin/makeblastdb -parse_seqids -dbtype nucl \
    -in $WD/internal_data/alpaca/imgt_alpaca_ig_d.fasta \
    -out $WD/internal_data/alpaca/alpaca_D


# J segment database
$WD/bin/edit_imgt_file.pl \
$DATA/vdj_gapped/imgt_gapped_alpaca_IGHJ.fasta > $WD/internal_data/alpaca/imgt_alpaca_ig
_j.fasta


$WD/bin/makeblastdb -parse_seqids -dbtype nucl \
    -in $WD/internal_data/alpaca/imgt_alpaca_ig_j.fasta \
    -out $WD/internal_data/alpaca/alpaca_J


cp -R $DATA/internal_data/alpaca/. $WD/database
cp -R $DATA/internal_data/alpaca/. $DATA/database
#cp -R $DATA/internal_data/alpaca/. $WD/internal_data/alpaca
```

```
%%bash
#for Sample in {10_S1,11_S2,12_S3,13_S4,14_S5,15_S6,16_S7,17_S8,18_S9,19_S10,20_S11,21_S
12,22_S13,23_S14,24_S15,25_S16}; do \
#for Sample in {A-S1,B-S3,C-S5,D-S7,E-S9,F-S11,D-S2,A-S4,B-S6,E-S8,F-S10}; do \
#for Sample in {SSA-S1,SA-S1}; do \

#for Sample in {A2A_S1,L2A_S2,PDA_S3,GblockA_S4,NegA_S5,A2G1_S6,L2G1_S7,PD1G1_S8,PD2G1_S
9,GblockG1_S10,GloveG1_S11,A2G2_S12,L2G2_S13,PD1G2_S14,GblockG2_S15}; do \
for Sample in {25_S16,PhageDisplay,Alpaca,Gblock,Alex,Lara}; do \

#ID=A2G1
#Index=6

#Sample="$ID"_S"$Index"
#Sample=10_S1


WD=/home/troyhinkley/ncbi-igblast-1.17.1
DATA=/home/troyhinkley/alpaca/igblast
SEQ="$Sample"_atleast-2.fasta
DATA_DIR=/home/troyhinkley/alpaca/EVD68

IN=catout
OUT=catout

cd $WD

bin/igblastn \
    -germline_db_V database/alpaca_V \
    -germline_db_D database/alpaca_D \
    -germline_db_J database/alpaca_J \
    -auxiliary_data optional_file/alpaca_gl.aux \
    -domain_system imgt -ig_seqtype Ig -organism alpaca \
    -outfmt '7 std qseq sseq btop' \
    -query $DATA_DIR/$IN/$SEQ \
    -out $DATA_DIR/$OUT/$Sample.fmt7

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
     MakeDb.py igblast \
    -i data/$OUT/$Sample.fmt7 \
    -s data/$IN/$SEQ \
    -r data/igblast/vdj_gapped/imgt_gapped_alpaca_IGHV.fasta /data/igblast/vdj_gapped/im
gt_gapped_alpaca_IGHD.fasta /data/igblast/vdj_gapped/imgt_gapped_alpaca_IGHJ.fasta \
    --outdir /data/$OUT \
    --failed \
    --log /data/log.txt \
    --format airr \
    --extended

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseDb.py select -d data/$OUT/"$Sample"_db-pass.tsv \
    -f productive -u T --regex --outname $Sample
```

```
sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseDb.py select -d data/$OUT/"$Sample"_parse-select.tsv \
    -f v_call -u IGHV --regex -o data/$OUT/$Sample-RepOut.tsv

done
```

Part 2 Notes:

1. It appears that this step is related to creating an International Immunogenetics Information System (IMGT (https://imgt.org/)) database. Note that IMGT provides reference directories for normal and pathological Immunoglobulin (Ig) and T-Cell Receptors (TR or TCRs).
2. For helpful information, see: Parsing IMGT Output (https://changeo.readthedocs.io/en/stable/examples/imgt.html).

This part has been skipped for now

Part 3) SCOPer - Spectral Clustering-Based Method for Identifying B Cell Clones

SCOPer (https://scoper.readthedocs.io/en/stable/) provides a computational framework for the identification of B cell clonal relationships from Adaptive Immune Receptor Repertoire sequencing (AIRR-Seq) data.

SCOPer includes methods for:

1. Assigning clonal identifiers using sequence identity
2. Hierarchical clustering
3. Spectral clustering

A key step in the quantitative analysis of AIRR-Seq data is the identification of B cell clones (that is, sequences derived from a common ancestor) using computational approaches.

`scoper` provides computational tools for the identification of B cell clones from AIRR-Seq data among immunoglobulin/B cell receptor sequences which share the same V gene, J gene, and junction length. With `scoper`, we can infer clonal relationships with predefined clustering thresholds, or an adaptive threshold.

Installation and Environment Setup

We install SCOPer in R via its CRAN Repository (https://cran.rstudio.com/web/packages/scoper/)

```
# this just says: if loading the package returns false, install it (else load the packag
e)
if (!require("scoper")) install.packages("scoper")
```

Note that this may not work without explicitly installing `Biostrings`, which can be installed using Bioconductor (https://bioconductor.org/packages/release/bioc/html/Biostrings.html) with the commands below:

```
if (!require("BiocManager")) install.packages("BiocManager")

# Now we use Bioconductor
BiocManager::install("Biostrings")
```

We learn that we will need GenomicAlignments (https://bioconductor.org/packages/release/bioc/html/GenomicAlignments.html), as well:

```
BiocManager::install("GenomicAlignments")
```

Clonal Partitioning

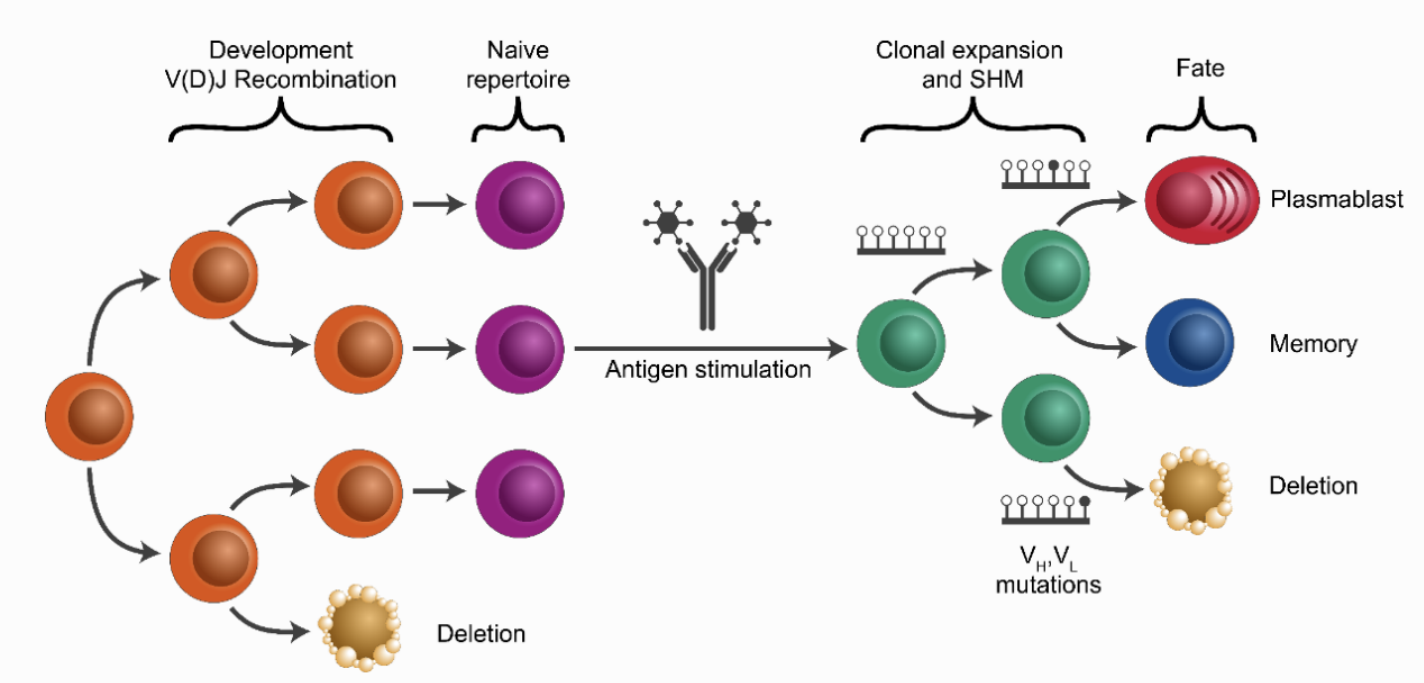
Note: As with previous sections, this relies heavily on the Immcantation documentation. In this case, we use the SCOPer Vignette (https://scoper.readthedocs.io/en/stable/vignettes/Scoper-Vignette/).

Goals of Analysis

Goal: Partition (cluster) sequences into clonally related lineages. Each lineage is a group of sequences that came from the same original naive cell.

Summary of the key steps:

1. etermine clonal clustering threshold: sequences which are under this cut-off are clonally related.
2. Assign clonal groups: add an annotation (clone_id) that can be used to identify a group of sequences that came from the same original naive cell.
3. Reconstruct germline sequences: figure out the germline sequence of the common ancestor, before mutations are introduced during clonal expansion and SMH.
4. Analize clonal diversity: number and size of clones? any expanded clones?



Example Data

We rely on a subset of Ig sequencing data from Larserson and Vigneault, et al., *PNAS* (2014), which includes sequences from multiple time-points before and after the subject received an immunization.

To identify clones, we require the following columns in the table:

- `junction`
- `c_call`
- `j_call`
- `sequence_alignment`
- `germline_alignment_d_mask`

The example data can be accessed in R by loading the package, and then accessing the dataframe:

```
library(scoper)

# Look at the top-6 entries
head(scoper::ExampleDb)
```

Hierarchical Clones

Most current studies uses a more sophisticated definition for clonal relationships. These studies leverage the high diversity of the junction region as a fingerprint to identify each B cell clone. Since it is unlikely that two separate recombination events would lead to identical junctions, sequences with junction regions that are similar enough are determined to share a common B cell ancestor (i.e., be clonally related) rather than to have arisen independently. Hierarchical clustering is a widely used distance-based method for identify clonally related sequences. An implementation of the hierarchical clustering approach is provided via the `hierachicalClones` function.

It is important to determine an appropriate threshold for trimming the hierarchical clustering into B cell clones before using this method. The ideal threshold for separating clonal groups is the value that separates the two modes of the nearest-neighbor distance distribution. The nearest-neighbor distance distribution can be generated by using the `distToNearest` function in the shazam R package. The threshold itself can be found using the `findThreshold` function in the same package. The resulting distribution should be bimodal, with the first mode representing sequences with clonal relatives in the dataset and the second mode representing singletons. For further details regarding inferring an appropriate threshold for the hierarchical clustering method, see the Distance to Nearest Neighbor (https://shazam.readthedocs.io/en/stable/vignettes/DistToNearest-Vignette) vignette in the shazam package.

- Technical details can be found in: Gupta NT, et al. (2017). Hierarchical clustering can identify B cell clones with high confidence in Ig repertoire sequencing data. *The Journal of Immunology*

Note that identifying clonal groups using `hierachicalClones` is nearly the same as the approach using the `identicalClones` function, with the additional requirement of a distance threshold.

The results `data.frame` , summary plots, and summary table are available. This will show the minimum inter (between) and maximum intra (within) clonal distances along with an effective threshold, which is an explanatory value calculated to represent the cut-off separating the inter and intra clonal distances. The effective threshold may differ from clustering threshold provided as input to hierarchicalClones.

```
# Define our database as the example database
DB <- scoper::ExampleDb

# Note that distToNearest is from shazam
db1 <- shazam::distToNearest(DB, model="ham", normalize="len",
                             vCallColumn="v_call", jCallColumn = "j_call",
                             sequenceColumn = "junction")

# Calculate threshold
threshold <- shazam::findThreshold(db1$dist_nearest,
                                   method = "gmm", model="gamma-norm",
                                   cutoff="intersect")
# Round off
thr <- round(threshold@threshold, 3)

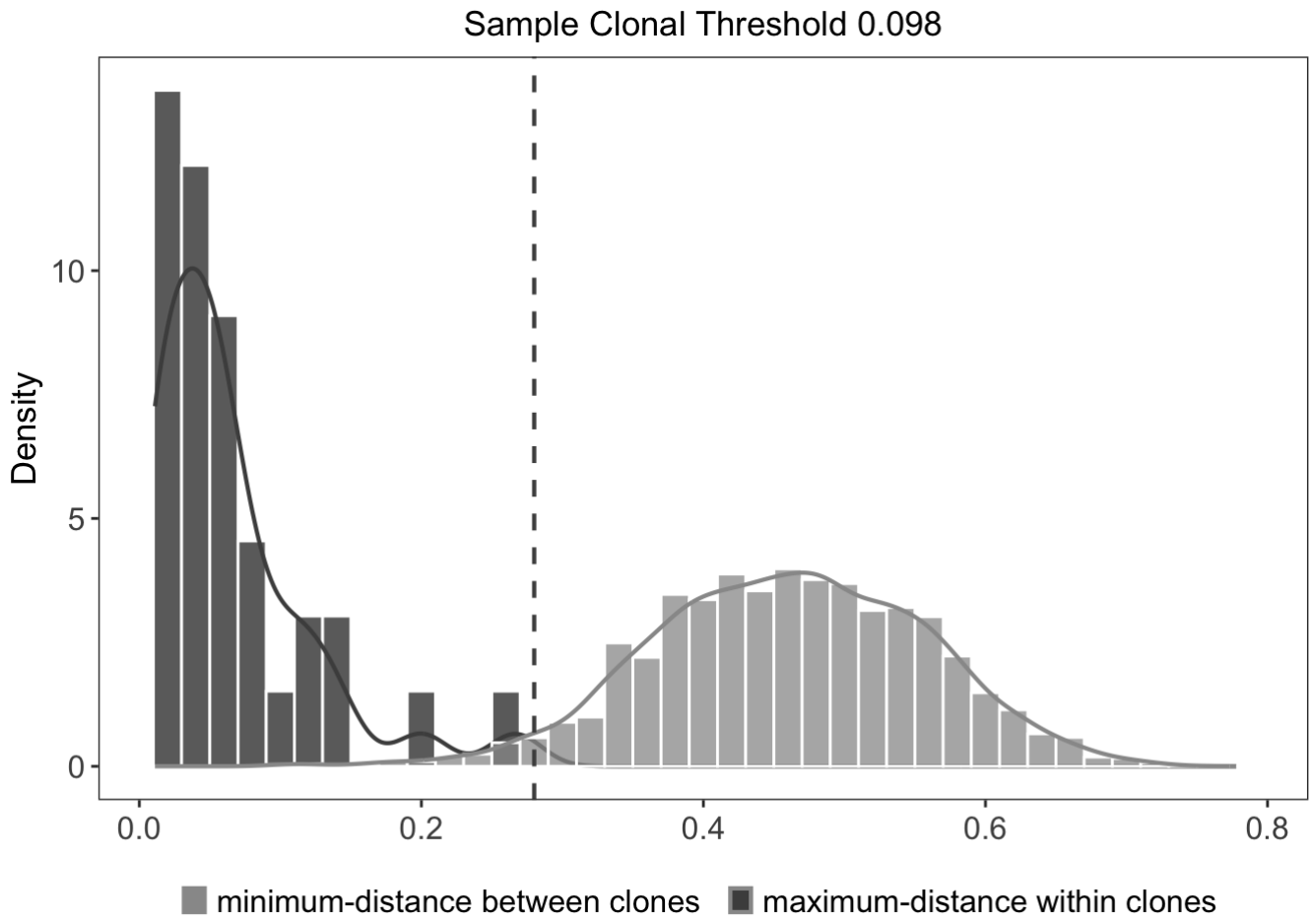
# Clonal assignment using hierarchical clustering
results <- scoper::hierarchicalClones(DB, method="nt", threshold=thr)
```

```
## Running defineClonesScoper in bulk mode and only keep heavy chains
```

```
## ... identifying heavy chains with getLocus(v_call).
```

```
# Graph
scoper::plotCloneSummary(results, binwidth=0.02, title=paste("Sample","Clonal Threshol
d", thr))
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## ℹ Please use `after_stat(density)` instead.
## ℹ The deprecated feature was likely used in the scoper package.
##   Please report the issue at <https://bitbucket.org/kleinstein/scoper/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
# Note that the plotted threshold line is not identical to the calculated threshold line
```

Clone Identification by Spectral Clustering

While the hierarchical clustering method groups sequences using a fixed distance supervised threshold, the spectral clustering-based model uses an adaptive unsupervised threshold to tune the required level of similarity among sequences in different local neighborhoods. It can be used as an alternative if the nearest-neighbor distance distribution is unimodal, meaning that `findThreshold` wasn't able to find the threshold at which to cut the hierarchy.

There are two available spectral clustering methods provided by the spectralClones function:

1. `method="novj"` : Infers clonal relationships using an adaptive threshold that indicates the level of similarity among junction sequences in a local neighborhood
2. `method="vj"` : Infers clonal relationships not only based on the junction region homology, but also taking into account the mutation profiles in the V and J segments

It is not mandatory, but a fixed threshold can also be provided to `spectralClone` s (threshold argument) which will enforce an upper-limit cut-off. When specifying the threshold argument, any sequence with distances above the threshold value from all sequences will become a singleton. The threshold can be defined as discussed above using nearest-neighbor distance distribution methods provided in the shazam. If `method="vj"` , the threshold will be applied on the distances calculated combining junction region homology and V/J mutation profiles.

Technical details can be found in:

- Nouri N and Kleinstein SH (2018). A spectral clustering-based method for identifying clones from high-throughput B cell repertoire sequencing data. *Bioinformatics*
- Nouri N and Kleinstein SH (2020). Somatic hypermutation analysis for improved identification of B cell clonal families from next-generation sequencing data, *PLoS Comput Biol*

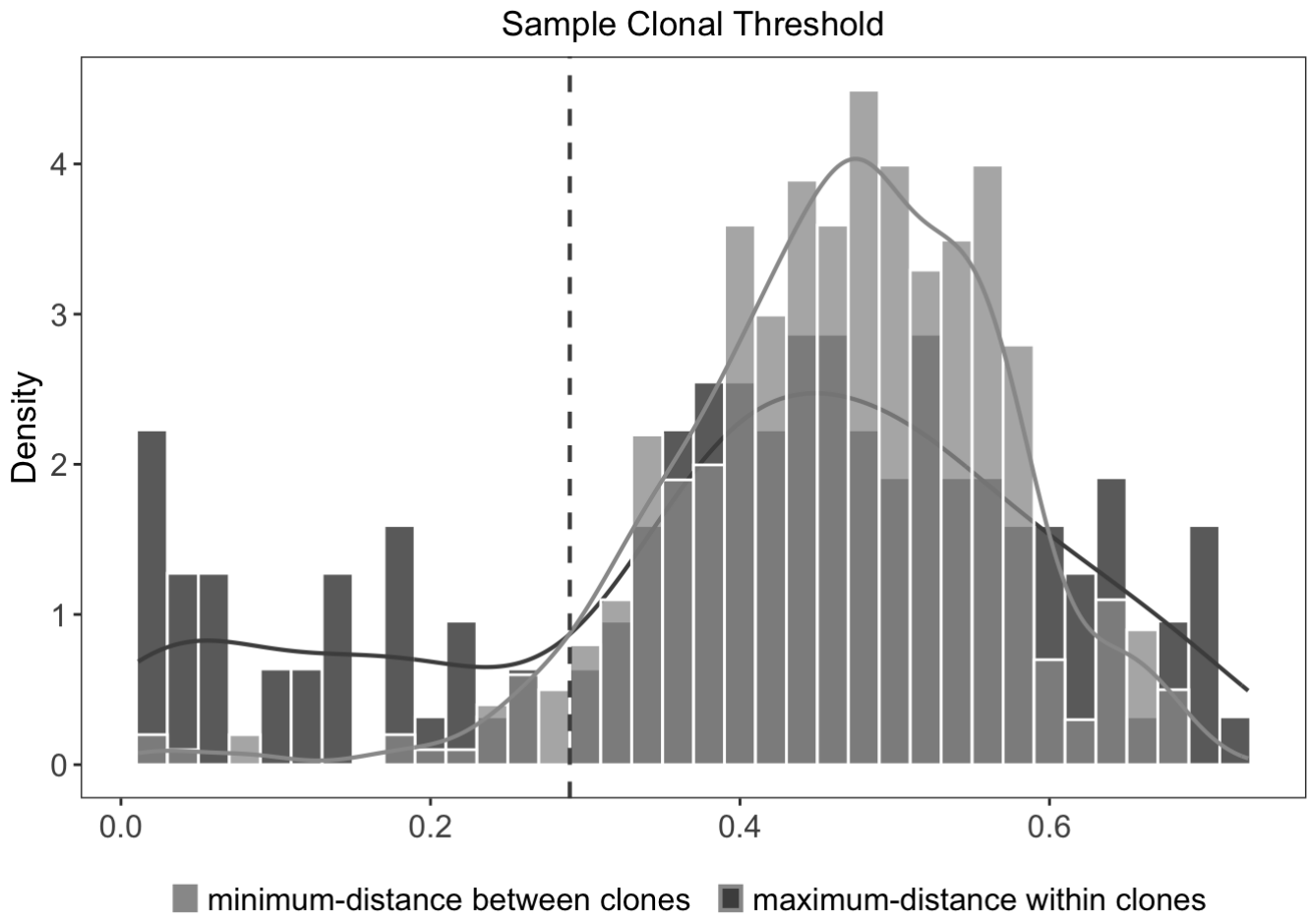
```
# Define our database as the example database
DB <- scoper::ExampleDb

# This is the method TH's code used
results <- scoper::spectralClones(DB, method="vj", nproc=16, cdr3=TRUE, iter_max=200, ns
tart = 50)
```

```
## Running defineClonesScoper in bulk mode and only keep heavy chains
```

```
## ... identifying heavy chains with getLocus(v_call).
```

```
# Plot
scoper::plotCloneSummary(results, binwidth=0.02, title=paste("Sample","Clonal Threshol
d"))
```



Note that we can save these data with:

```
# Convert results to data.frame
results <- as.data.frame(results)

# Write to YAML or JSON
# the default behavior references the output file string's extension
# airr::write_airr(results, file = "~/file/path")
```

Part 4) Create Germlines

Note that a helpful reference is here:

- Introduction to B cell repertoire analysis using the Immcantation framework (https://immcantation.readthedocs.io/en/stable/tutorials/intro-lab.html)

```
DATA_DIR=$(pwd)
#for Sample in {A-S1,B-S3,C-S5,D-S7,E-S9,F-S11,D-S2,A-S4,B-S6,E-S8,F-S10}; do \
#for Sample in {A-S1,B-S3,C-S5,D-S7,E-S9,F-S11}; do \
#for Sample in {SA-S1,SSA-S1} ; do \
#for Sample in {10_S1,11_S2,12_S3,13_S4,14_S5,15_S6,16_S7,17_S8,18_S9,19_S10,20_S11,21_S
12,22_S13,23_S14,24_S15,25_S16}; do \
#for Sample in {A2A_S1,L2A_S2,PDA_S3,GblockA_S4,A2G1_S6,L2G1_S7,PD1G1_S8,PD2G1_S9,Gblock
G1_S10,GloveG1_S11,A2G2_S12,L2G2_S13,PD1G2_S14,GblockG2_S15}; do \
#for Sample in {NegA_S5,A2G1_S6,L2G1_S7}; do \

Sample=GblockA_S4

Sub0=vdj_gapped
Sub1=EVD68/spectralclones
Sub2=EVD68/fullgerm1

IN=data/$Sub1/$Sample-clones.tsv
DB0=data/$Sub1/$Sample-0.tsv
DB1=data/$Sub1/$Sample-1.tsv
DB2=data/$Sub1/$Sample-2.tsv
DB3=data/$Sub1/$Sample-3.tsv
OUT=data/$Sub2/$Sample-fullgerm.tsv

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    CreateGermlines.py -d $IN -g full dmask \
        -r data/$Sub0 -o $DB0 --cloned --cf clone_id

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseDb.py add -d $DB0  -f sample -u $Sample -o $DB1

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseDb.py sort -d $DB1  -f duplicate_count --num --descend -o $DB2

sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
    ParseDb.py index -d $DB2 -o $OUT -f index

#sudo docker run -v $DATA_DIR:/data:z immcantation/suite:devel \
#    ParseDb.py drop -d $DB3 -o $OUT -f sequence_id

rm $(pwd)/$Sub1/$Sample-0.tsv
rm $(pwd)/$Sub1/$Sample-1.tsv
rm $(pwd)/$Sub1/$Sample-2.tsv
rm $(pwd)/$Sub1/$Sample-3.tsv

#done
```

Part 5) Infer SHM

Note that this part relies heavily on:

- SHazaM Targeting Vignette (https://shazam.readthedocs.io/en/stable/vignettes/Targeting-Vignette/)

SHazaM

SHazaM (https://shazam.readthedocs.io/en/stable/) (CRAN Documentation (https://cran.r-project.org/web/packages/shazam/shazam.pdf)) is part of the Immcantation analysis framework for Adaptive Immune Receptor Repertoire sequencing (AIRR-seq) and provides tools for advanced analysis of somatic hypermutation (SHM) in immunoglobulin (Ig) sequences. Shazam focuses on the following analysis topics:

1. **Quantification of mutational load** – SHazaM includes methods for determine the rate of observed and expected mutations under various criteria. Mutational profiling criteria include rates under SHM targeting models, mutations specific to CDR and FWR regions, and physicochemical property dependent substitution rates.
2. **Statistical models of SHM targeting patterns** - Models of SHM may be divided into two independent components: (a) a mutability model that defines where mutations occur and (b) a nucleotide substitution model that defines the resulting mutation. Collectively these two components define an SHM targeting model. SHazaM provides empirically derived SHM 5-mer context mutation models for both humans and mice, as well tools to build SHM targeting models from data.
3. **Analysis of selection pressure using BASELINe** - The Bayesian Estimation of Antigen-driven Selection in Ig Sequences (BASELINe) method is a novel method for quantifying antigen-driven selection in high-throughput Ig sequence data. BASELINe uses SHM targeting models can be used to estimate the null distribution of expected mutation frequencies, and provide measures of selection pressure informed by known AID targeting biases.
4. **Model-dependent distance calculations** - SHazaM provides methods to compute evolutionary distances between sequences or set of sequences based on SHM targeting models. This information is particularly useful in understanding and defining clonal relationships.

SHM Targeting Models

The targeting model is the background likelihood of a particular mutation, based on the surrounding sequence context as well as the mutation itself. The model is inferred from observed mutations in the data. The model can then be transformed into a distance function to compare Ig sequences of a given dataset based on the likelihood of the observed mutations. This is done via the following steps:

1. Infer a substitution model, which is the likelihood of a base mutating to each other base given the microsequence context.
2. Infer a mutability model, which is likelihood of a given base being mutated given the microsequence context and substitution model.
3. Visualize the mutability model to identify hot and cold spots.
4. Calculate a nucleotide distance matrix based on the underlying SHM models.

Example Data

A small example AIRR Rearrangement database is included in the `alakazam` package. Inferring a targeting model requires the following fields (columns) to be present in the table:

- `sequence_id`
- `sequence_alignment`
- `germline_alignment_d_mask`
- `v_call`

```
# Load example data
library(shazam)
```

```
## Loading required package: ggplot2
```

```
## As of v1.0.0 the AIRR Rearrangement schema is now the default file format.
## A description of the standard is available at https://docs.airr-community.org.
## The legacy Change-O format is supported through arguments to each function
## that allow the input column names to be explicitly defined.
```

```
data(ExampleDb, package="alakazam")

# Subset to IGHG for faster usage demonstration
db <- subset(ExampleDb, c_call == "IGHG")
```

Infer targeting model(substitution and mutability)

The function for inferring substitution rates (`createSubstitutionMatrix`) counts the number of mutations from a given base to all others occurring in the center position for all 5-mer motifs in the dataset.

The `model` argument of `createSubstitutionMatrix` allows the user to specify whether to count all mutations, or just silent mutations to infer the model. Column names for the sample sequence, germline sequence, and V call can also be passed in as parameters if they differ from Change-O defaults.

Additionally, the `multipleMutation` parameter determines handling of multiple mutations in a single 5-mer: `independent` treats each mutation independently and ignore entirely disregards `5-mers` with multiple mutations.

```
# Create substitution model using silent mutations
sub_model <- shazam::createSubstitutionMatrix(db, model="s",
                                              sequenceColumn="sequence_alignment",
                                              germlineColumn="germline_alignment_d_mas
k",
                                              vCallColumn="v_call")
```

The function for inferring a mutability model (`createMutabilityMatrix`) counts the number of mutations in all 5-mer motifs of the dataset, and depends upon the inferred substitution rates. Similar parameters as those available for inferring the substitution rates are available to adjust this function.

```
# Create mutability model using silent mutations
mut_model <- shazam::createMutabilityMatrix(db, sub_model, model="s",
                                            sequenceColumn="sequence_alignment",
                                            germlineColumn="germline_alignment_d_mask",
                                            vCallColumn="v_call")
```

```
## Warning in shazam::createMutabilityMatrix(db, sub_model, model = "s",
## sequenceColumn = "sequence_alignment", : Insufficient number of mutations to
## infer some 5-mers. Filled with 0.
```

`createMutabilityMatrix` creates an object of class `MutabilityModel` that contains a named numeric vector of 1024 normalized mutability. The numbers of silent and replacement mutations used for estimating the 5-mer mutabilities are recorded in the `numMutS` and `numMutR` slots, respectively. The `source` slot contains a named vector indicating whether each 5-mer mutability was inferred or measured. A data.frame with both the mutability values and derivation source.

```
# Number of silent mutations used for estimating 5-mer mutabilities
mut_model@numMutS
# Number of replacement mutations used for estimating 5-mer mutabilities
mut_model@numMutR
# Mutability and source as a data.frame
head(as.data.frame(mut_model))
```

The inferred substitution and mutability models returned by the above functions only account for unambiguous 5-mers. However, there may be cases in which the user may need the likelihood of a mutation in a 5-mer with ambiguous characters. Each of the above functions has a corresponding function (`extendSubstitutionMatrix` and `extendMutabilityMatrix`) to extend the models to infer 5-mers with Ns by averaging over all corresponding unambiguous 5-mers.

```
# Extend models to include ambiguous 5-mers
sub_model <- shazam::extendSubstitutionMatrix(sub_model)
mut_model <- shazam::extendMutabilityMatrix(mut_model)
```

These extended substitution and mutability models can be used to create an overall SHM targeting matrix (`createTargetingMatrix`), which is the combined probability of mutability and substitution.

```
# Create targeting model matrix from substitution and mutability models
tar_matrix <- shazam::createTargetingMatrix(sub_model, mut_model)
```

All of the above steps can be combined by using the single function `createTargetingModel` to infer a `TargetingModel` object directly from the dataset. Again, the numbers of silent and replacement mutations used for estimating the 5-mer mutabilities are also recorded in the `numMutS` and `numMutR` slots respectively.

Additionally, it is generally appropriate to consider the mutations within a clone only once. Consensus sequences for each clone can be generated using the `collapseClones` function.

```
# Collapse sequences into clonal consensus
clone_db <- shazam::collapseClones(db, cloneColumn="clone_id",
                                   sequenceColumn="sequence_alignment",
                                   germlineColumn="germline_alignment_d_mask",
                                   nproc=1)

# Create targeting model in one step using only silent mutations
# Use consensus sequence input and germline columns
model <- shazam::createTargetingModel(clone_db, model="s", sequenceColumn="clonal_sequen
ce",
                                      germlineColumn="clonal_germline", vCallColumn="v_c
all")
```

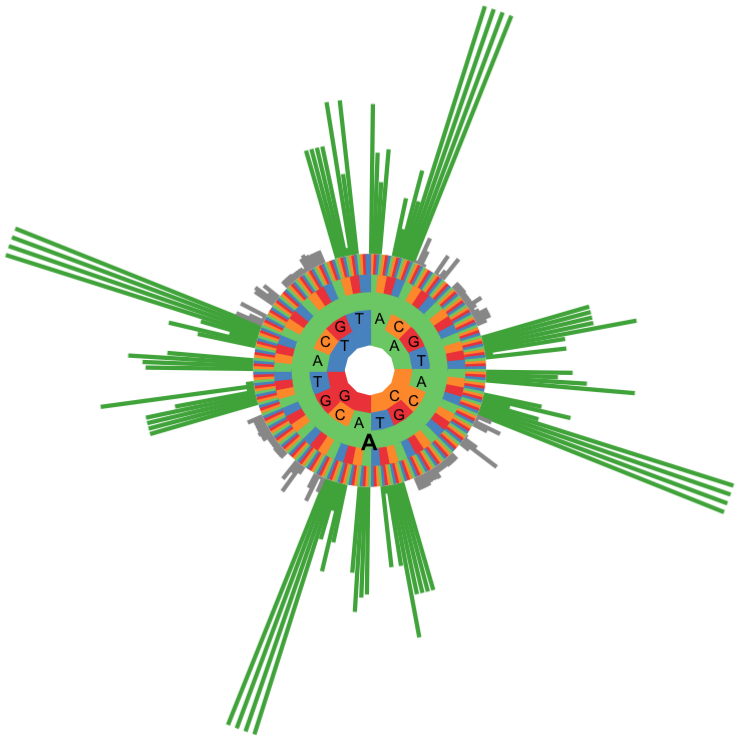
```
## Warning in createMutabilityMatrix(db, sub_mat, model = model, sequenceColumn =
## sequenceColumn, : Insufficient number of mutations to infer some 5-mers. Filled
## with 0.
```

Targeting Model Visualization

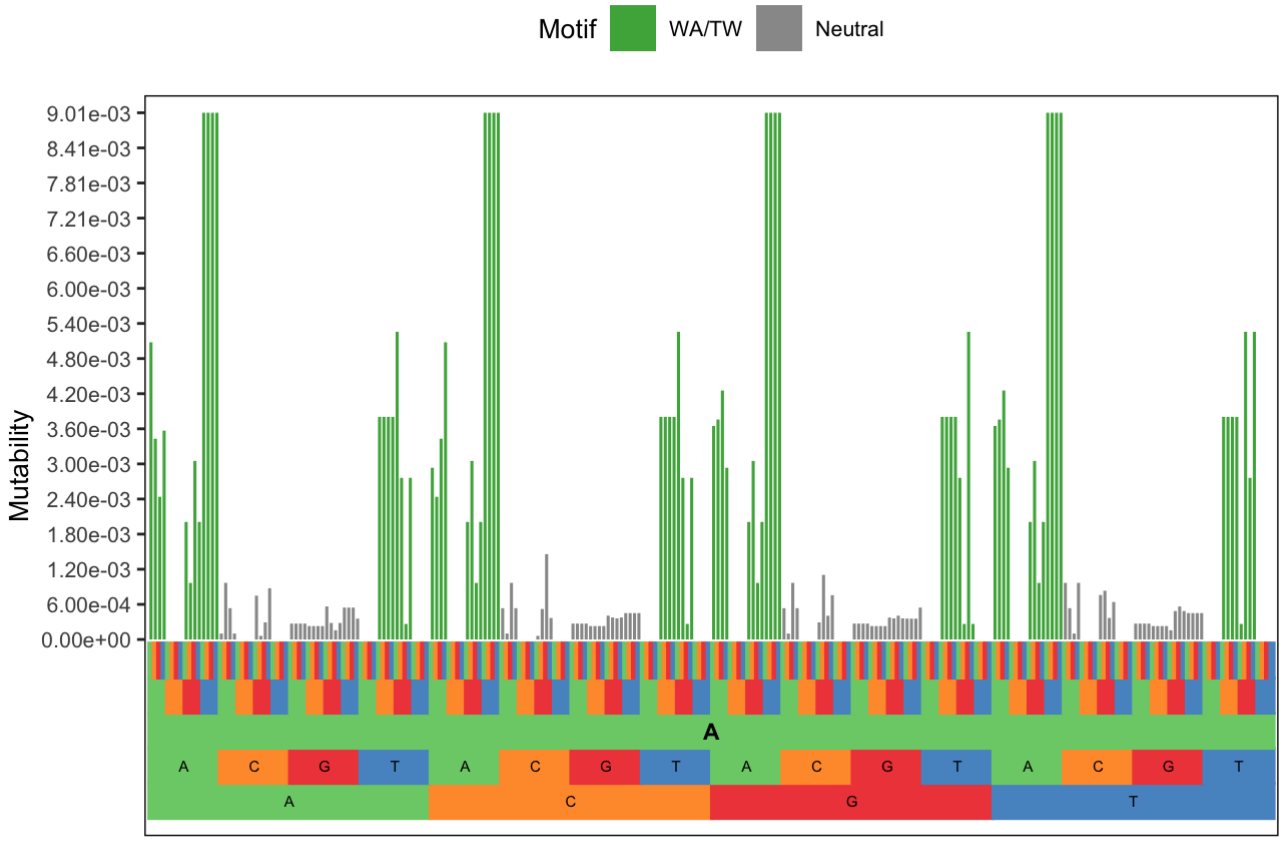
The visualization of a dataset's underlying SHM mutability model can be used to investigate hot and cold spot motifs. The length of the bars on the plot of mutability rates corresponds to the likelihood of a given base in the given 5-mer being mutated.

The plotting function `plotMutability` has an argument `style` to specify either a hedgehog plot (circlular) or barplot diplay of 5-mer mutability rates. If the mutability for only specific bases is required, this can be specified via the `nucleotides` argument.

```
# Generate hedgehog plot of mutability model
shazam::plotMutability(model, nucleotides="A", style="hedgehog")
```




```
# Generate bar plot of mutability model
shazam::plotMutability(model, nucleotides="A", style="bar")
```



Calculate targeting distance matrix

In the Change-O pipeline, the `hs5f` cloning method rely on an inferred targeting model. If users wish to use a targeting model inferred from their data to assign distance between sequences for clonal grouping, then the observed SHM targeting rates must be transformed into distances. The `calcTargetingDistance` function returns a matrix of distances between each 5-mer and each corresponding mutation of the center base. This matrix can also be generated and written directly to a file using the function `writeTargetingDistance` .

```
# Calculate distance matrix
dist <- shazam::calcTargetingDistance(model)
```